Chapter 19 — Cheat Sheet

19.1 Architecture Decision Matrix

Conceptual Deep Dive

Need	Start with	Escalate to	Evidence to escalate
Better outputs	Clearer prompt, examples	Reasoning model / CoT	Eval gap on multi-step tasks
Private/changing knowledge	RAG (hybrid + rerank)	GraphRAG / agentic RAG	Multi-hop or global questions fail
Strict format	Structured outputs	Grammar-constrained decoding	Repair rate high
External actions	Function calling	Agent with checkpoints	Path can't be fixed in advance
Style / behaviour / cost	Few-shot	LoRA SFT, then DPO	Prompt can't reach bar
Volume cost	Smaller model / cache	Distil + self-host	Break-even volume reached
Latency	Stream, prompt cache	Smaller model, speculative decoding	p95 over SLA
Safety	Least privilege + validation	Guardrail classifiers, human approval	Red-team findings

Cross-Connection Scenarios

Use with Chapter 16 system-design framework.

Production Anti-Patterns

- Skipping rows: jumping to the right column without evidence.

Minimal Working Code

```
def recommend(task: dict) -> str:
    if task.get("knowledge_changes") or task.get("private_docs"):
        return "RAG (hybrid + reranker)"
    if task.get("needs_actions") and task.get("path_unknown"):
        return "agent with budgets and approvals"
    if task.get("needs_actions"):
        return "workflow + function calling"
    if task.get("volume_high") and task.get("narrow"):
        return "distill/fine-tune small model"
    return "prompt + few-shot + structured output"

print(recommend({"private_docs": True}))
```

19.2 Formula Sheet

Conceptual Deep Dive

Quantity	Formula
Softmax with temperature	p_i = exp(z_i/T) / Σ exp(z_j/T)
Cross-entropy	-Σ y log p ; perplexity = exp(CE)
Attention	softmax(QK^T/√d_k)V
Cosine similarity	a·b / (||a||||b||)
Euclidean vs cosine (unit vectors)	||a–b||^2 = 2 – 2cos(a,b)
RRF	Σ 1/(k + rank), k≈60
BM25	Σ idf(q)·tf·(k1+1)/(tf + k1(1–b+b·|d|/avgdl))
nDCG	DCG/IDCG, DCG = Σ gain_i / log2(i+1)
MRR	mean(1/rank_first_relevant)
LoRA	W' = W + (α/r)·B·A
DPO loss	–log σ(β[(logπ_w – logπ_ref,w) – (logπ_l – logπ_ref,l)])
Weight memory	params × bits/8 bytes
KV cache	2 × layers × kv_heads × head_dim × seq × batch × bytes
Training FLOPs	≈ 6 × params × tokens
Chinchilla tokens	≈ 20 × params
Decode tokens/s (upper bound)	memory bandwidth / model bytes
Exponential backoff (full jitter)	sleep = U(0, min(cap, base·2^n))
Token-bucket	tokens = min(cap, tokens + Δt·rate)
A/B sample size / arm	≈ (z_α+z_β)^2·[p1(1–p1)+p2(1–p2)] / Δ^2
Compounded agent reliability	p^steps

Cross-Connection Scenarios

Chapters 5, 8, 10, 12, 13 derive and use these.

Production Anti-Patterns

- Using the formulas as exact rather than planning estimates.

Minimal Working Code

```
import math

def perplexity(ce: float) -> float:
    return math.exp(ce)

def agent_reliability(p_step: float, steps: int) -> float:
    return p_step ** steps

print(perplexity(2.0), agent_reliability(0.95, 10))
```

19.3 Tool Comparison Tables

Conceptual Deep Dive

Vector stores

Tool	Hosting	Hybrid	Filtering	Scale	Pick when
pgvector	Self/managed PG	Via FTS	SQL	~10M	Already on Postgres
Qdrant	Self/cloud	Yes	Strong payload	100M+	Filter-heavy
Milvus	Self/cloud	Yes	Good	Billions	Huge scale
Pinecone	Managed	Yes	Good	Large	No ops
FAISS	Library	No	Manual	In-memory	Research/embedded

Inference engines

Engine	Strength	Weakness
vLLM	Throughput, ecosystem	GPU/Linux focus
SGLang	Prefix reuse, structured output	Smaller ecosystem
TensorRT-LLM	Peak NVIDIA perf	Build complexity
TGI	HF integration	Check maintenance status
llama.cpp / Ollama	Local, CPU/Metal	Not for high concurrency

Quantisation

Method	Bits	Best for
GPTQ	3–4	GPU serving
AWQ	4	GPU serving, quality
GGUF k-quants	2–8	CPU/Apple/edge
bitsandbytes NF4	4	QLoRA training
FP8	8	H100+ serving

Agent frameworks

Framework	Style
LangGraph	Explicit graph, durable state
AutoGen / MS Agent Framework	Conversational multi-agent
CrewAI	Role-based crews
PydanticAI / OpenAI Agents SDK / Claude Agent SDK	Lightweight typed agent loops
DSPy	Programmatic prompt optimisation

Eval and observability

Tool	Focus
RAGAS	RAG metrics, synthetic sets
TruLens	RAG triad, feedback functions
DeepEval	pytest-style LLM tests
promptfoo	Prompt matrices, red teaming
Phoenix / Langfuse / LangSmith	Traces, datasets, scores

Cross-Connection Scenarios

Chapters 8, 10, 11, 12, 13, 14.

Production Anti-Patterns

- Choosing from a table without testing on your workload.

Minimal Working Code

```
STORES = {"pgvector": {"ops": 1, "scale"
: 2}, "qdrant": {"ops": 2, "scale": 4},
         "milvus": {"ops": 4, "scale": 5}, "pinecone": {"op
s": 0, "scale": 4}}

def pick(max_ops: int, min_scale: int) -> list[str]:
    return [k for k, v in STORES.items() if v["ops"] <= max_ops and v["scale"] >
;= min_scale]

print(pick(max_ops=2, min_scale=3))
```

19.4 Quick Syntax Reference

Conceptual Deep Dive

- **asyncio:** await asyncio.gather(*coros, return_exceptions=True); async with asyncio.Semaphore(n); asyncio.timeout(s).
- **httpx:** async with httpx.AsyncClient(timeout=30) as c: r = await c.post(url, json=body).
- **FastAPI SSE:** StreamingResponse(gen(), media_type="text/event-stream").
- **Pydantic v2:** Model.model_validate_json(s), Model.model_json_schema().
- **PyTorch loop:** opt.zero_grad(); loss.backward(); opt.step().
- **pgvector:** ORDER BY emb <=> $1 LIMIT k (cosine), <-> L2, <#> negative inner product.
- **vLLM:** vllm serve MODEL --max-model-len N --tensor-parallel-size G --enable-prefix-caching.
- **Docker GPU:** docker run --gpus all -p 8000:8000 IMAGE.
- **Kubernetes GPU:** resources.limits["nvidia.com/gpu"]: 1.
- **LoRA (PEFT):** LoraConfig(r=16, lora_alpha=32, target_modules=[...]).
- **OTel GenAI attrs:** gen_ai.request.model, gen_ai.usage.input_tokens, gen_ai.usage.output_tokens.

Cross-Connection Scenarios

Pair with each chapter's full examples.

Production Anti-Patterns

- Copy-pasting syntax without error handling.

Minimal Working Code

```
import asyncio
import httpx

async def main():
    sem = asyncio.Semaphore(5)
    async with httpx.AsyncClient(timeout=30) as c:
        async def get(u):
            async with sem:
                return (await c.get(u)).status_code
        print(await asyncio.gather(get("https://example.com"), return_exceptions=True))

if __name__ == "__main__":
    asyncio.run(main())
```

19.5 Debugging Cheat Sheet: Symptom to Likely Cause

Conceptual Deep Dive

Symptom	Likely causes	First check
Output cut off mid-sentence or invalid JSON	max_tokens too low, finish_reason=length	Log finish_reason
Model ignores instruction	Contradictory prompt, buried instruction, too many rules	Move to top and end; simplify
Answers fine in playground, bad in app	Different system prompt, template, params, or truncated context	Diff the rendered request
RAG answers wrong but sources exist	Chunk lost context, rank too low, noisy context	Check recall@k per stage
RAG says "no info" for covered topic	Query/doc vocabulary gap, filter too strict	Try hybrid + multi-query
Repeated retries then outage	Retry storm, no jitter or breaker	Check retry budget
429s under load	TPM exceeded, burst, per-process limiter	Token-aware shared limiter
Streaming arrives in one lump	Proxy buffering	Disable buffering, check headers
Fine-tuned model never stops	EOS/template mismatch	Inspect training labels
Fine-tune worse than base	Bad data, too many epochs, LR too high	Eval at checkpoints
vLLM OOM at startup	max_model_len or utilisation too high, weights + KV > VRAM	Lower length, quantise
High p99, fine p50	Long prompts, queueing, cold cache	Break down by length and queue time
Cost spike	Longer prompts, loops, retries, model change	Group spend by feature/model
Agent loops	No stop condition, errors trimmed from context	Loop detector, step cap
Cross-user data leak	Missing ACL filter, shared cache key, memory scope	Audit keys and filters

Cross-Connection Scenarios

Each row maps to chapters: 2.5, 4, 7, 9.7, 11.7, 12.7, 13.8, 14.

Production Anti-Patterns

- Changing multiple things at once while debugging.
- Debugging from memory of the prompt instead of the logged, rendered request.

Minimal Working Code

```
RULES = [
    (lambda e: e.get("finish_reason") == "length", "output truncated: raise max_
tokens"),
    (lambda e: e.get("status") == 429, "rate limited: token-aware shared limiter + backoff
"),
    (lambda e: e.get("retrieved", 1) == 0, "empty retrieval: check filters / query rewriti
ng"),
    (lambda e: e.get("repair_rate", 0) > 0.1, "high schema-repair rate: constrain decod
ing / simplify schema"),
]

def diagnose(event: dict) -> list[str]:
    return [msg for cond, msg in RULES if cond(event)] or ["no known pattern; inspect the trace"
;]
```

```
print(diagnose({"finish_reason": "length", "retrieved": 0}))
```

```
def diff_requests(a: dict, b: dict, path: str = "") -> li
st[str]:
    """Find what differs between the playground request and the app request."&q
uot;
    diffs = []
    for k in sorted(set(a) | set(b)):
        pa, pb = a.get(k), b.get(k)
        if isinstance(pa, dict) and isinstance(pb, dict):
            diffs += diff_requests(pa, pb, f"{path}{k}.")
        elif pa != pb:
            diffs.append(f"{path}{k}: {str(pa)[:40]!r} != {str(pb)[:40]!r}")
    return diffs

print(diff_requests({"temperature": 0.2, "model": "m"}, {"temperature&
quot;: 1.0, "model": "m"}))
```

19.6 Numbers to Remember (Order-of-Magnitude Estimates)

Conceptual Deep Dive

Treat as planning heuristics, not facts; re-measure for your stack.

Quantity	Rule of thumb
English text	about 0.75 words per token, about 4 characters per token
Embedding storage	`N x d x 4` bytes (fp32); 1M x 1024 dims = about 4 GB
Weight memory	7B at fp16 about 14 GB; 4-bit about 3.5-4 GB; 70B at 4-bit about 35-40 GB
KV cache per token (7B-class, GQA)	order of 100-500 KB per token at fp16
Decode speed bound	memory bandwidth / model bytes
Typical chunk size	200-500 tokens, 10-20% overlap
Retrieval depth	retrieve 50-100, rerank, pass 3-10
RRF constant	k = 60
LoRA rank	8-64; LR 1e-4 to 2e-4
DPO beta	0.1-0.5
Backoff	base 0.5-1 s, cap 20-60 s, 3-5 attempts
Interactive TTFT target	under 1-2 s; voice end-to-end under 0.8 s
Eval set size	start 50-100, grow to several hundred; CI width about 1/sqrt(n)
Agent reliability	0.95 per step over 10 steps is about 0.60
Capacity headroom	30-40% above peak; keep GPUs under about 85% utilisation for strict SLOs

Cross-Connection Scenarios

Back-of-envelope math in system design interviews (16.3) and capacity planning (13.7).

Production Anti-Patterns

- Quoting these as guarantees instead of measuring.

Minimal Working Code

```
def quick_estimates(params_b: float, bits: int, n_vectors: int, dims:
int, kv_kb_per_tok: float,
                    ctx: int, concurrent: int) -> dict:
    return {
        "weights_gb": round(params_b * bits / 8, 1),
        "vectors_gb": round(n_vectors * dims * 4 / 1e9, 2),
        "kv_gb": round(kv_kb_per_tok * 1e3 * ctx * concurrent / 1e9, 1),
    }

print(quick_estimates(8, 4, 5_000_000, 768, 130, 4096, 16))
```

```
import math

def ci_halfwidth(p: float, n: int, z: float = 1.96) -> float:
    return z * math.sqrt(p * (1 - p) / n)

for n in (50, 200, 1000):
    print(n, round(ci_halfwidth(0.8, n), 3))
```

19.7 Glossary of Core Terms

Conceptual Deep Dive

- **Agent**: an LLM in a loop that chooses actions (tool calls) toward a goal.
- **ANN**: approximate nearest neighbour search (HNSW, IVF, DiskANN).
- **Attention**: mechanism mixing token information using query-key-value similarity.
- **BM25**: lexical ranking function using term frequency, document length and rarity.
- **BPE**: byte-pair encoding tokenisation by iterative merges.
- **Chain-of-thought**: eliciting intermediate reasoning steps.
- **Chunking**: splitting documents into retrievable units.
- **Context window**: maximum tokens the model can process in one request.
- **Continuous batching**: scheduling at token-iteration level to keep the GPU full.
- **Cross-encoder**: model scoring a (query, document) pair jointly; used for reranking.
- **DPO**: preference optimisation without a separate reward model.
- **Embedding**: vector representation capturing semantic similarity.
- **Faithfulness / groundedness**: answer claims are supported by provided context.
- **Fine-tuning**: further training to change model behaviour.
- **Function calling**: model emits structured requests to run tools.
- **Guardrail**: check or control around inputs, outputs or actions.
- **Hallucination**: fluent output unsupported by evidence or facts.
- **HNSW**: hierarchical navigable small-world graph index.
- **HyDE**: retrieve using an embedding of a hypothetical answer.
- **Idempotency**: repeating an operation has the same effect as doing it once.
- **KV cache**: stored attention keys/values from earlier tokens.
- **LLM-as-judge**: using a model to grade outputs against a rubric.
- **LoRA / QLoRA**: low-rank adapter fine-tuning (QLoRA on a 4-bit base).
- **MCP**: Model Context Protocol for connecting models to tools and data.
- **MoE**: mixture-of-experts architecture routing tokens to expert MLPs.
- **nDCG / MRR / recall@k**: ranking and retrieval metrics.
- **PEFT**: parameter-efficient fine-tuning.
- **PagedAttention**: block-based KV cache memory management.
- **Prefill / decode**: prompt processing phase / token-by-token generation phase.
- **Prompt caching**: reusing computation for a repeated prompt prefix.
- **Prompt injection**: untrusted text that hijacks model instructions.
- **Quantisation**: storing weights in fewer bits.
- **RAG**: retrieval-augmented generation.
- **Reranker**: second-stage model reordering retrieval candidates.
- **RLHF / RLVR**: reinforcement learning from human feedback / verifiable rewards.
- **RRF**: reciprocal rank fusion of ranked lists.
- **Speculative decoding**: draft-and-verify acceleration.
- **SFT**: supervised fine-tuning.

- **Structured output**: model output constrained to a schema.
- **Temperature / top-p / top-k / min-p**: sampling controls.
- **TTFT / ITL**: time to first token / inter-token latency.
- **Vector database**: store optimised for similarity search over embeddings.

Cross-Connection Scenarios

Every term is defined in context in its chapter; use this list for quick recall before interviews.

Production Anti-Patterns

- Using terms loosely with stakeholders (for example calling every retrieval failure a "hallucination").

Minimal Working Code

```
GLOSSARY = {
    "ttft": "time to first token",
    "itl": "inter-token latency",
    "rrf": "reciprocal rank fusion",
    "kv cache": "stored attention keys/values from earlier tokens",
}

def define(term: str) -> str:
    return GLOSSARY.get(term.lower(), "not in glossary")

print(define("RRF"))
```

```
def quiz(terms: dict[str, str], seed: int = 0, n: int = 3):
    import random
    rng = random.Random(seed)
    for t in rng.sample(list(terms), min(n, len(terms))):
        yield t, terms[t]

print(list(quiz(GLOSSARY, n=2)))
```

19.8 Prompt Pattern Library

Conceptual Deep Dive

Reusable skeletons (fill the angle-bracket slots, then evaluate on your data): 1. **Classifier**: "Classify into exactly one of: . If none fit, answer unknown. Output only the label." 2. **Extractor**: "Extract from . Use null when absent. Include the supporting quote for each field. Return JSON matching ." 3. **Grounded QA**: "Answer using only the sources. Cite as [n]. If the sources do not contain the answer, say you do not know." 4. **Rewriter**: "Rewrite for in . Preserve all numbers, names and meaning. Do not add information." 5. **Judge**: "Given , think step by step, then output JSON {reasoning, score, verdict}. Be strict." 6. **Planner**: "Break into at most N steps. For each: action, tool, expected result, success check." 7. **Critic/reviser**: "Review against . List concrete problems, then produce a corrected version." 8. **Query rewriter**: "Rewrite the last user message as a standalone search query using the conversation." 9. **Safe refusal**: "Decline briefly, name no policy text, offer one safe alternative." 10. **Few-shot formatter**: "\nExamples:\n.....\nNow: {x}" Edge cases: patterns interact with model families (some prefer XML tags, others markdown), and examples must reflect real distribution including hard and negative cases.

Cross-Connection Scenarios

Prompt engineering (6), structured output (7), grounded QA (9.5), judges (10.3).

Production Anti-Patterns

- Copy-pasting patterns without evaluating them on your inputs.
- Stacking all ten patterns into one giant prompt.

Minimal Working Code

```
PATTERNS = {
    "classifier": "Classify the input into exactly one of: {labels}. If none fit, answer u
nknown. Output only the label.\nInput: {text}",
    "grounded_qa": "Answer using only the sources below. Cite as [n]. If unsupported, say
you do not know.\n{sources}\nQuestion: {question}",
    "rewriter": "Rewrite for {audience} in a {tone} tone. Preserve all numbers and names.
Add nothing.\nText: {text}",
}

def render(name: str, **kw) -> str:
    return PATTERNS[name].format(**kw)

print(render("classifier", labels="billing, bug, other", text="I was charged twi
ce"))
```

```
def validate_pattern_vars(name: str, kw: dict) -> list[str]:
    import string
    needed = {f for _, f, _, _ in string.Formatter().parse(PATTERNS[name]) if f}
    return sorted(needed - kw.keys())

print(validate_pattern_vars("rewriter", {"audience": "executives"}))
```

19.9 Rapid-Fire Review Questions

Conceptual Deep Dive

Answer aloud in under 30 seconds each; answers are in the referenced sections. - Why divide attention scores by the square root of d_k? (5.4) - What dominates KV-cache size and how does GQA help? (5.4) - When would you choose HNSW over IVF-PQ? (8.3) - Why is RRF robust to score-scale differences? (8.5) - Name three signs your retrieval, not your generator, is failing. (9.7) - How do you reduce position bias in pairwise judging? (10.3) - Why should retries use jitter and budgets? (4.3) - What makes an operation safe to retry? (4.5) - Why is decode memory-bandwidth bound while prefill is compute-bound? (13.1) - What is the LoRA update equation and what do r and alpha control? (12.3) - Why can DPO skip the reward model? (12.4) - Give two defences against indirect prompt injection that do not rely on prompt wording. (6.5, 14.2) - How do you size an A/B test and avoid peeking? (10.5) - Why prefer workflows to agents when the path is known? (11.1) - What belongs in an AI incident postmortem? (16.6) - How would you compute cost per successful task? (17.12) Edge cases: if you cannot answer, re-read the section and then re-implement its code example from memory.

Cross-Connection Scenarios

Use with interview preparation (16.7) and mastery checkpoints (18).

Production Anti-Patterns

- Memorising answers without being able to apply them to a system design.

Minimal Working Code

```
QUESTIONS = [
    ("Why scale attention by sqrt(d_k)?", "5.4"),
    ("What makes an operation safe to retry?", "4.5"),
    ("Why does RRF ignore raw scores?", "8.5"),
    ("What does the KV cache store?", "5.4"),
]

def schedule(results: dict[str, bool]) -> list[str]:
    """Review wrong answers first."""
    return [q for q, _ in QUESTIONS if not results.get(q, False)] + [q for q, _ in QUESTIONS if results.g
```

```
et(q, False)]

print(schedule({"What does the KV cache store?": True})[0])
```

```
def coverage_by_chapter(asked: list[str]) -> dict[str, int]:
    from collections import Counter
    sec = {q: s.split(".")[0] for q, s in QUESTIONS}
    return dict(Counter(sec[q] for q in asked if q in sec))

print(coverage_by_chapter([q for q, _ in QUESTIONS]))
```